



Full-stack software development

TypeScript

R. Naudts, J. Pieck, J. Rombouts, B. Van Impe

JavaScript

Recap

Variables

```
// difference let and const
let username = `John`;
username = `Marthe`;
const age = 17;

// Concatenation with template literals
let greeting = `Hello ${username}, ${age} year old`;

// Conditional ternary operator
const state = age < 18 ? `Child` : `Adult`; // Child
```

Functions

```
// Functions
const calculateSum = (a, b) => {
  return a + b;
}
const multiply = (a, b) => a * b; // Functional programming

// Functions can be treated like variables: First-class functions
let other = calculateSum;
console.log(other(2, 3)); // "5"

// Higher-order functions (Callback function): pass function as parameter
const calculate = (a, b, operation) => operation(a, b);
const s = calculate(2, 3, calculateSum); // 5
const m = calculate(2, 3, multiply); // 6
```

Objects

```
// Objects
const identity = { name: `John`, age: 45 };
console.log(`Hello ${identity.name} who is ${identity.age} years old.`);

// Destructuring object fields in variables
const { myName, myAge } = identity // myName=`John`, myAge=45
```

Arrays

```
//array of objects
let persons = [{ name: `John`, age: 45 }, { username: `Annie`, age: 23 }]

//looping over array
for (const person of persons) {
  console.log(`Hello ${username}, ${age} year old`);
}

// Spreading & destructuring
let males = [`John`, `Frank`];
let females = [`Annie`, `Martha`];
let people = [...males, ...females]
    //result: [`John`, `Frank`, `Annie`, `Martha`]
const [john, frank] = males; //Result: john = `John`, frank=`Frank`
```

ForEach and Filter as higher-order functions

```
const greet = (person) => console.log(`Hello ` + person);  
persons.forEach(greet);  
  
// inline as anonymous function  
persons.forEach((person) => console.log(`Hello ${person}`));  
  
persons.filter((person) => person.age > 25); //John
```



TypeScript

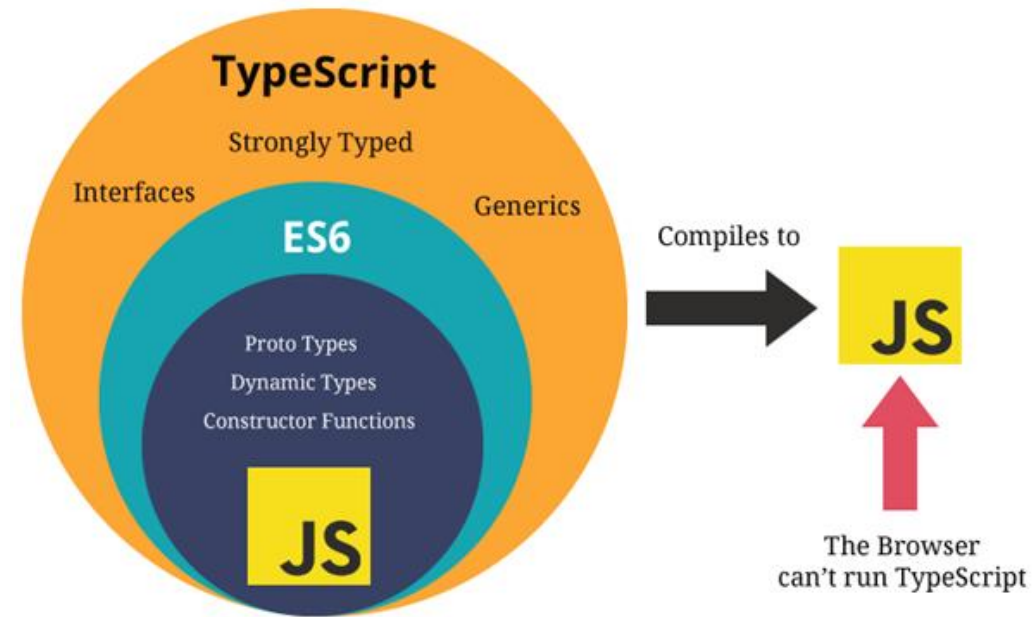
TypeScript

- Is JavaScript with syntax for types
- Is a strongly typed programming language that builds on JavaScript, giving you better tooling at any scale
- Is a compiled programming language: the TypeScript Compiler (TSC) must convert the instructions written in TypeScript to its JavaScript equivalent

	JavaScript	TypeScript
Typing	Loosely typed. In JavaScript, only dynamic typing is supported, not static typing.	Strongly Typed. TypeScript supports both static and dynamic typing .
Compilation	JavaScript does not need a compilation.	TypeScript needs to be compiled . Typescript code is compiled to JavaScript code.
Error Detection	In JavaScript, errors are detected only at the run time, since it is an interpreted language.	Errors are detected or highlighted in the early development stage in TypeScript.
Type	JavaScript is only a scripting language that gives functionality to the pages. There is no support for interfaces.	TypeScript is an enhanced language with interfaces, OOP concepts, classes , static typing, etc.

How to Get Started with TypeScript

- Write the TypeScript code
- Compile the TypeScript code into plain JavaScript code
- Deploy the JavaScript code to any environment that runs JavaScript



Variables and parameters must be typed

JavaScript

```
let username = `John`;  
const age = 17;  
const pi = 3.14;  
  
const multiply = (a, b)  
  => a * b;
```

TypeScript

```
let firstName: string;  
  // firstName is declared as string  
let username = `John`;  
  // type by Inference after assignment  
const age = 17;  
const multiply = (a: number, b: number)  
  => a * b;
```

```
7  username = 7;  
demoAsTS.ts 1 of 3 problems  
type 'number' is not assignable to type 'string'. ts(2322)
```

Type Alias: Define Your Own Type

JavaScript

```
const john = { name: `John`, age: 45 };

console.log(`Hello ${john.name} who is ${john.age} years old.`);
```

TypeScript

```
type Person = {
  name: string;
  age: number;
}

const john: Person =
  { name: `John`, age: 45 };

console.log(`Hello ${john.name} who is ${john.age} years old.`);
```

Typing function parameters

JavaScript

```
const calculateSum = (a, b) => {  
  return a + b;  
}  
  
const multiply = (a, b) => a * b;
```

TypeScript

```
const calculateSum = (a: number, b: number): number => {  
  return a + b;  
}  
  
const multiply = (a: number, b: number) => a * b;
```

Decomposing a Typescript function

Function name	Parameter type	Return type	Function body
<code>const demo</code>	<code>= ({ x }: { x: number })</code>	<code>: string</code>	<code>=> `...`;</code>

Parameter type is an **object** that must contain a **field** "x" of the **type** number.

```
demo({ x: 2 }); // correct
demo(2); //incorrect
demo({ 2}); //incorrect
demo({ x: 2, name: `foo` })//incorrect
```

Parameter Destructuring: Unpack Objects

JavaScript

```
const { myName, myAge } = john

const sendMessage = (person) =>
  `Hello ${person.name},
    aged ${person.age}`;
```

TypeScript

```
type Person = { name: string; age: number; }

// destructure function parameter
const sendMessage = ({ name, age }: Person): string
=> `Hello ${name}, aged ${age}`;

// destructure function parameter type
const hi = (person: { name: string, age: number }): string =>
`Hello ${person.name}, aged ${person.age}`;

// without destructuring
const hello = (person: Person): string =>
`Hello ${person.name}, aged ${person.age}`;

// put the value of the field name in new parameter myName
const john: Person = { name: `John`, age: 45 };
const { name: myName, age: myAge } = john
console.log(myName); //John
```


Class JavaScript

```
const emma = { name: `Emma`, age: 25 };  
  
console.log(`Hello ${emma.name} who is ${emma.age}  
years old.`);
```

TypeScript

```
class Identity {  
  name: string;  
  age: number;  
  
  constructor(identity: { name: string, age: number }) {  
    this.name = identity.name;  
    this.age = identity.age;  
  }  
  
  equals(otherUser: { name: string, age: number }): boolean  
  {  
    return this.name === otherUser.name  
      && this.age === otherUser.age;  
  }  
}  
  
const emma: Identity =  
  new Identity({ name: `Emma`, age: 25 });  
  
console.log(`Hello ${emma.name} who is ${emma.age} years old.`);
```

Generic Classes

JavaScript

```
const john = { name: `John`, age: 45 };  
  
const persons = [john]
```

TypeScript

```
type Person = {  
  name: string;  
  age: number;  
}  
  
const john: Person = { name: `John`, age: 45 };  
  
const persons: Array<Person> =[john]
```

? (optional parameters)

TypeScript

```
class Animal {  
  name: string;  
  age?: number;  
  constructor(animal: { name: string, age?: number }) {  
    this.name = animal.name;  
    this.age = animal.age;  
  }  
}  
  
const dog = new Animal({ name: `Shae`, age: 3 });  
const cat = new Animal({ name: `Minou` });
```

<https://www.typescriptlang.org/docs/handbook/2/objects.html#optional-properties>
<https://www.typescriptlang.org/docs/handbook/2/functions.html#optional-parameters>

Enums

- An **enum** (enumeration) is a feature that allows you to define a set of named constants.

```
enum Role { STUDENT, LECTURER }
```

Modules, Export default

Used to export a single value, object or function from a module (e.g. counter.db.ts), that can be imported in another module.

```
// counter.service.ts
...
export default {getAllCounters, getCounterById}
```



```
// counter.routes.ts
import counterService from './services/counter.service.ts';
...
counterService.getAllCounters();
counterService.getCounterById();
```



```
// You can use any name for the importing the default exported value
import anyName from './services/counter.service.ts';
...
anyName.getAllCounters();
anyName.getCounterById();
```

Composition

TypeScript

```
class Profile {
  firstname: string
  lastname: string
  email: string
}

class User {
  username: string
  password: string
  profile: Profile

  constructor(user: {username:string, password:string, profile:Profile}) {
    this.username = user.username
    this.password = user.password
    this.profile = user.profile
  }
}

const elke : User = new User({username:"Elke", password:"t", profile: {firstname: "Elke", lastname:"Steegmans", email:"elke@uc11.be"}})
```

References

- <https://www.typescriptlang.org>
 - <https://www.typescriptlang.org/docs/handbook/typescript-in-5-minutes.html>
- <https://livecodestream.dev/post/what-why-and-how-of-typescript-for-javascript-developers/>
- <https://www.interviewbit.com/blog/typescript-vs-javascript/>
- <https://www.linkedin.com/pulse/typescript-superset-javascript-sunil-sharma/>